Stats 21 - HW 3

Jackelin Hernandez

The questions have been entered into this document. You will modify the document by entering your code.

Make sure you run the cell so the requested output is visible. Export it as an HTML file and then print to PDF.

Homework is an opportunity to practice coding and to practice problem solving.

Doing exercises is where you will do most of your learning.

Using AI or copying someone else's solutions takes away your learning opportunities. It is also academic dishonesty.

Reading

- Chapter 10 - Dictionaries
- Chapter 12 - Tuples
- Chapter 13 - Files

Please keep up with the reading. The chapters are short.

Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.1

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

Write a function that reads the words in `words.txt` and stores them as keys in a dictionary. It doesn't matter what the values are. Then you can use the `in` operator as a fast way to check whether a string is in the dictionary.

In [1]:
```
word_dict = {}
def make_word_dict():
    tex = open('words.txt')
    for line in tex:
        word = line.strip().lower()
        word_dict[word] = 1
    pass
make_word_dict()
```

In [2]:
```
"hello" in word_dict
```

Out[2]: True

Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.4

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

In HW2 Exercise 10.7, you created a function called `has_duplicates()`. It takes a list as a parameter and returns `True` if there is any object that appears more than once in the list.

Use a dictionary to write a faster, simpler version of `has_duplicates()`.

In [3]:
```
def has_duplicates(t):
    dic = {}
    for letter in t:
        if letter not in dic:
            dic[letter] = 1
        else:
            return True
    return False
```

In [4]:
```
has_duplicates(['a','b','b','c'])
```

Out[4]: True

In [5]:
```
has_duplicates(['a','b','c','a'])
```

Out[5]: True

Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.5

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

A Caesar cipher is a weak form of encryption that involves 'rotating' each letter by a fixed number of places. To rotate a letter means to shift it through the alphabet, wrapping around to the beginning if necessary. "A" rotated by 3 is "D". "Z" rotated by 1 is "A".

Two words are "rotate pairs" if you can rotate one of them and get the other. For example, "cheer" rotated by 7 is "jolly".

Write a script that reads in the wordlist `words.txt` and finds all the rotate pairs of words that are 5 letters or longer.

One function that could be helpful is the function `ord()` which converts a character to a numeric code. Keep in mind that numeric codes for uppercase and lowercase letters are different.

Some hints:

- you can write helper functions, such as a function that will rotate a letter by a certain number and/or another function that will rotate a word by a number of letters
- to keep your script running quickly, you should use the wordlist dictionary from exercise 11.1

In [6]:
```
let_places = {97:'a', 98:'b', 99:'c', 100:'d', 101:'e', 102:'f', 103:'g', 104:'h', 105:'i',
              106:'j', 107:'k', 108:'l', 109:'m', 110:'n', 111:'o', 112:'p', 113:'q', 114:'r',
              115:'s', 116:'t', 117:'u', 118:'v', 119:'w', 120:'x', 121:'y', 122:'z'}
```

```
def rotate(word, places):
    while places > 122:
        places -= 122
    new_lett = []

    for letter in word.lower():
        adj = ord(letter) + places

        if adj > 122:
            adj -= 26

        new_lett.append(let_places[adj])

    return ''.join(new_lett)

for word in word_dict:
    if len(word) >= 5:
        for place in range(1, 26):
            rot_word = rotate(word, place)
            if rot_word in word_dict:
                print(f''' {word}, {rot_word}''')
```

abjurer, nowhere
adder, beefs
ahull, gnarr
alkyd, epoch
alula, tenet
ambit, setal
anteed, bouffe
aptly, timer
arena, river
baiza, tsars
banjo, ferns
beefs, adder
benni, ruddy
biffs, holly
bolls, hurry
bombyx, hushed
bouffe, anteed
bourg, viola
buffi, hallo
bulls, harry
bunny, sleep
butyl, hazer
chain, ingot
cheer, jolly
clasp, raphe
cogon, sewed
commy, secco
corky, wiles
craal, penny
credo, shute
creel, perry
cubed, melon
curly, wolfs
cushy, wombs
danio, herms
dated, spits
dawted, splits
dazed, spots
didos, pupae
dolls, wheel
drips, octad
ebony, uredo
eches, kinky
epoch, alkyd
fadge, torus
fagot, touch
ferns, banjo
fills, lorry
fizzy, kneed
frena, wiver
frere, serer
fusion, layout
ganja, kerne
gassy, smeek
ginny, motte
gnarl, xeric
gnarr, ahull
golem, murks
golly, murre
green, terra
gulfs, marly
gulls, marry

gummy, masse
gunny, matte
hallo, buffi
harry, bulls
hazer, butyl
herms, danio
hints, styed
hoggs, tasse
holly, biffs
hotel, ovals
hurry, bolls
hushed, bombyx
ingot, chain
inkier, purply
jerky, snath
jiffs, polly
jimmy, posse
jinni, potto
jinns, potty
johns, punty
jolly, cheer
kerne, ganja
kinky, eches
kneed, fizzy
lallan, pepper
later, shaly
latex, shale
layout, fusion
linum, rotas
lorry, fills
luffs, rally
lutea, pyxie
manful, thumbs
marly, gulfs
marry, gulls
masse, gummy
matte, gunny
melon, cubed
mills, sorry
mocha, suing
molas, surgy
motte, ginny
muffs, sally
mulch, sarin
mumms, sassy
munch, satin
murks, golem
murre, golly
muumuu, weewee
noggs, tummy
nowhere, abjurer
nulls, tarry
nutty, tazze
octad, drips
ovals, hotel
oxter, vealy
pecan, tiger
penny, craal
pepper, lallan
perry, creel
polly, jiffs
posse, jimmy

potto, jinni
potty, jinns
primero, sulphur
pulpy, varve
punty, johns
pupae, didos
purply, inkier
pyxie, lutea
rally, luffs
raphe, clasp
ratan, vexer
river, arena
rotas, linum
ruddy, benni
sally, muffs
sarin, mulch
sassy, mumms
satin, munch
secco, commy
serer, frere
setal, ambit
sewed, cogon
shale, latex
shaly, later
sheer, tiffs
shute, credo
sleep, bunny
smeek, gassy
snath, jerky
sneer, toffs
sorry, mills
spits, dated
splits, dawted
spots, dazed
steeds, tuffet
steer, tuffs
styed, hints
suing, mocha
sulphur, primero
surgy, molas
tarry, nulls
tasse, hoggs
tazze, nutty
teloi, whorl
tenet, alula
terra, green
thumbs, manful
tiffs, sheer
tiger, pecan
timer, aptly
toffs, sneer
torus, fadge
touch, fagot
tsars, baiza
tuffet, steeds
tuffs, steer
tummy, noggs
uredo, ebony
varve, pulpy
vealy, oxter
vexer, ratan
viola, bourg

```
weewee, muumuu
wheel, dolls
whorl, teloi
wiles, corky
wiver, frena
wolfs, curly
wombs, cushy
xeric, gnarl
```

In [7]:
```
has_duplicates(['a','b','b','c'])
```

Out[7]: True

Modified Exercise based on Think Python 2ed. Chap 12, Section 10, Exercise 12.2

https://greenteapress.com/thinkpython2/html/thinkpython2013.html

Write a program that reads a word list from a file (see Section 9.1) and finds all the sets of words that are anagrams.

After finding all anagram sets, print the list of all anagram sets that have 6 or more entries in it.

Here is an example of what the output might look like:

```
['abets', 'baste', 'bates', 'beast', 'beats', 'betas', 'tabes']
['acers', 'acres', 'cares', 'carse', 'escar', 'races', 'scare', 'serac']
['acred', 'arced', 'cadre', 'cared', 'cedar', 'raced']
...
```

Hint:

First traverse the entire wordlist to build a dictionary that maps from a collection of letters to a list of words that can be spelled with those letters. The question is, how can you represent the collection of letters in a way that can be used as a key? i.e. The word "eat" and the word "tea" should be in a list associated with a key ('a','e','t').

In [8]:
```
ana = {}

for word in word_dict:
    letters = tuple(sorted(tuple(word)))

    if letters in ana:
        ana[letters].append(word)
    else:
        ana[letters] = [word]

for words in ana:
    if len(ana[words]) >= 6:
        print(ana[words])
```

```
['abets', 'baste', 'bates', 'beast', 'beats', 'betas', 'tabes']
['acers', 'acres', 'cares', 'carse', 'escar', 'races', 'scare', 'serac']
['acred', 'arced', 'cadre', 'cared', 'cedar', 'raced']
['aiders', 'deairs', 'irades', 'raised', 'redias', 'resaid']
['airts', 'astir', 'sitar', 'stair', 'stria', 'tarsi']
['alert', 'alter', 'artel', 'later', 'ratel', 'taler']
['alerting', 'altering', 'integral', 'relating', 'tanglier', 'triangle']
['alerts', 'alters', 'artels', 'estral', 'laster', 'ratels', 'salter', 'slater', 'staler', 'stela
r', 'talers']
['alevin', 'alvine', 'valine', 'veinal', 'venial', 'vineal']
['algins', 'aligns', 'lasing', 'liangs', 'ligans', 'lingas', 'signal']
['aliens', 'alines', 'elains', 'lianes', 'saline', 'silane']
['aligners', 'engrails', 'nargiles', 'realigns', 'signaler', 'slangier']
['amens', 'manes', 'manse', 'means', 'mensa', 'names', 'nemas']
['anestri', 'nastier', 'ratines', 'retains', 'retinas', 'retsina', 'stainer', 'stearin']
['angriest', 'astringe', 'ganister', 'gantries', 'granites', 'ingrates', 'rangiest']
['apers', 'asper', 'pares', 'parse', 'pears', 'prase', 'presa', 'rapes', 'reaps', 'spare', 'spea
r']
['ardeb', 'barde', 'bared', 'beard', 'bread', 'debar']
['ardebs', 'bardes', 'beards', 'breads', 'debars', 'sabred', 'serdab']
['ares', 'arse', 'ears', 'eras', 'rase', 'sear', 'sera']
['aretes', 'easter', 'eaters', 'reseat', 'seater', 'teaser']
['aridest', 'astride', 'diaster', 'disrate', 'staider', 'tardies', 'tirades']
['aril', 'lair', 'liar', 'lira', 'rail', 'rial']
['ariled', 'derail', 'dialer', 'laired', 'railed', 'redial', 'relaid']
['arils', 'lairs', 'liars', 'liras', 'rails', 'rials']
['arles', 'earls', 'lares', 'laser', 'lears', 'rales', 'reals', 'seral']
['armets', 'master', 'maters', 'matres', 'ramets', 'stream', 'tamers']
['arrest', 'rarest', 'raster', 'raters', 'starer', 'tarres', 'terras']
['artiest', 'artiste', 'attires', 'iratest', 'ratites', 'striate', 'tastier']
['ashed', 'deash', 'hades', 'heads', 'sadhe', 'shade']
['aspen', 'napes', 'neaps', 'panes', 'peans', 'sneap', 'spean']
['aspers', 'parses', 'passer', 'prases', 'repass', 'spares', 'sparse', 'spears']
['aspirer', 'parries', 'praiser', 'rapiers', 'raspier', 'repairs']
['ates', 'east', 'eats', 'etas', 'sate', 'seat', 'seta', 'teas']
['bares', 'baser', 'bears', 'braes', 'saber', 'sabre']
['canter', 'centra', 'nectar', 'recant', 'tanrec', 'trance']
['canters', 'nectars', 'recants', 'scanter', 'tanrecs', 'trances']
['capers', 'crapes', 'escarp', 'pacers', 'parsec', 'recaps', 'scrape', 'secpar', 'spacer']
['caress', 'carses', 'crases', 'escars', 'scares', 'seracs']
['caret', 'carte', 'cater', 'crate', 'react', 'recta', 'trace']
['carets', 'cartes', 'caster', 'caters', 'crates', 'reacts', 'recast', 'traces']
['carpels', 'clasper', 'parcels', 'placers', 'reclasp', 'scalper']
['claroes', 'coalers', 'escolar', 'oracles', 'recoals', 'solacer']
['corset', 'coster', 'escort', 'rectos', 'scoter', 'sector']
['cruet', 'curet', 'cuter', 'eruct', 'recut', 'truce']
['cruets', 'cruset', 'curets', 'eructs', 'rectus', 'recuts', 'truces']
['dater', 'derat', 'rated', 'tared', 'trade', 'tread']
['dearths', 'hardest', 'hardset', 'hatreds', 'threads', 'trashed']
['deers', 'drees', 'redes', 'reeds', 'seder', 'sered']
['deils', 'delis', 'idles', 'isled', 'sidle', 'slide']
['deist', 'diets', 'dites', 'edits', 'sited', 'stied', 'tides']
['deltas', 'desalt', 'lasted', 'salted', 'slated', 'staled']
['deposit', 'dopiest', 'podites', 'posited', 'sopited', 'topside']
['diols', 'idols', 'lidos', 'sloid', 'soldi', 'solid']
['drapes', 'padres', 'parsed', 'rasped', 'spader', 'spared', 'spread']
['earings', 'erasing', 'gainers', 'reagins', 'regains', 'reginas', 'searing', 'seringa']
['easters', 'reseats', 'searest', 'seaters', 'teasers', 'tessera']
['easting', 'eatings', 'ingates', 'ingesta', 'seating', 'teasing']
['elastin', 'entails', 'nailset', 'salient', 'saltine', 'tenails']
['emits', 'items', 'metis', 'mites', 'smite', 'stime', 'times']
['empires', 'emprise', 'epimers', 'imprese', 'premies', 'premise', 'spireme']
```

```
['enosis', 'eosins', 'essoin', 'noesis', 'noises', 'ossein', 'sonsie']
['enters', 'nester', 'rentes', 'resent', 'tenser', 'ternes']
['esprit', 'priest', 'ripest', 'sprite', 'stripe', 'tripes']
['esprits', 'persist', 'priests', 'spriest', 'sprites', 'stirpes', 'stripes']
['ester', 'reest', 'reset', 'steer', 'stere', 'terse', 'trees']
['esters', 'reests', 'resets', 'serest', 'steers', 'steres']
['estrange', 'grantees', 'greatens', 'negaters', 'reagents', 'sergeant']
['estrin', 'inerts', 'insert', 'inters', 'niters', 'nitres', 'sinter', 'triens', 'trines']
['estrous', 'oestrus', 'ousters', 'sourest', 'souters', 'stoures', 'tussore']
['eviler', 'levier', 'liever', 'relive', 'revile', 'veiler']
['hales', 'heals', 'leash', 'selah', 'shale', 'sheal']
['hassel', 'hassle', 'lashes', 'selahs', 'shales', 'sheals']
['hectors', 'rochets', 'rotches', 'tochers', 'torches', 'troches']
['lapse', 'leaps', 'pales', 'peals', 'pleas', 'salep', 'sepal', 'spale']
['lavers', 'ravels', 'salver', 'serval', 'slaver', 'velars', 'versal']
['laves', 'salve', 'slave', 'vales', 'valse', 'veals']
['leapt', 'lepta', 'palet', 'petal', 'plate', 'pleat']
['least', 'setal', 'slate', 'stale', 'steal', 'stela', 'taels', 'tales', 'teals', 'tesla']
['leasts', 'slates', 'stales', 'steals', 'tassel', 'teslas']
['luster', 'lustre', 'result', 'rustle', 'sutler', 'ulster']
['lusters', 'lustres', 'results', 'rustles', 'sutlers', 'ulsters']
['mates', 'meats', 'satem', 'steam', 'tames', 'teams']
['merits', 'mister', 'miters', 'mitres', 'remits', 'smiter', 'timers']
['nestor', 'noters', 'stoner', 'tenors', 'tensor', 'toners', 'trones']
['notes', 'onset', 'seton', 'steno', 'stone', 'tones']
['opts', 'post', 'pots', 'spot', 'stop', 'tops']
['painters', 'pantries', 'pertains', 'pinaster', 'pristane', 'repaints']
['palest', 'palets', 'pastel', 'petals', 'plates', 'pleats', 'septal', 'staple']
['palters', 'persalt', 'plaster', 'platers', 'psalter', 'stapler']
['parties', 'pastier', 'piaster', 'piastre', 'pirates', 'traipse']
['parts', 'prats', 'sprat', 'strap', 'tarps', 'traps']
['paste', 'pates', 'peats', 'septa', 'spate', 'tapes', 'tepas']
['paster', 'paters', 'prates', 'repast', 'tapers', 'trapes']
['peers', 'peres', 'perse', 'prees', 'prese', 'speer', 'spree']
['peris', 'piers', 'pries', 'prise', 'ripes', 'speir', 'spier', 'spire']
['petrous', 'posture', 'pouters', 'proteus', 'spouter', 'troupes']
['piles', 'plies', 'slipe', 'speil', 'spiel', 'spile']
['realist', 'retails', 'saltier', 'saltire', 'slatier', 'tailers']
['recepts', 'respect', 'scepter', 'sceptre', 'specter', 'spectre']
['reigns', 'renigs', 'resign', 'sering', 'signer', 'singer']
['reins', 'resin', 'rinse', 'risen', 'serin', 'siren']
['resaw', 'sawer', 'sewar', 'sware', 'swear', 'wares', 'wears']
['staw', 'swat', 'taws', 'twas', 'wast', 'wats']
['stow', 'swot', 'tows', 'twos', 'wost', 'wots']
```

NumPy Exercises

For the following exercises, be sure to print the desired output. You will not receive credit for problems that do not print the desired output.

Some of these exercises will require the use of functions in numpy that may not have been covered in class. Look up documentation on how to use them. I always recommend checking the official reference at https://numpy.org/doc/stable/reference/index.html

In [9]:
```
import numpy as np
rng = np.random.default_rng(seed = 1)
```

Exercise NP.1:

Task: Create an array `b` of 10 random integers selected between 0-99

Desired output: `[47 51 75 95  3 14 82 94 24 31]` of course yours might be different

In [10]:
```
b = rng.integers(0, 100, size=10)
```

In [11]:
```
print(b)
```

```
[47 51 75 95  3 14 82 94 24 31]
```

NP.1b:

Task: reverse the elements in `b`. Hint: Try slicing the array, but backwards

In [12]:
```
b = b[range(-1, -11, -1)]
print(b)
```

```
[31 24 94 82 14  3 95 75 51 47]
```

NP.2a:

Task: Create an array `c` of 1000 random values selected from a normal distribution centered at 100 with sd = 15, rounded to 1 decimal place. Print only the first 10 values.

Desired output: `[ 92.1  83.9 113.   65.5 126.2  88.6 104.8  96.3 121.9  69.1]` of course your values may be different

In [13]:
```
c = rng.normal(100, 15, 1000).round(1)
print(c[0:10])
```

```
[106.7  91.9 108.7 105.5 104.4 100.4 108.2  89.   97.6  92.8]
```

NP.2b:

Perform a Shapiro-Wilk test to see if the values in c come from a normal distribution. Report the p-value and appropriate conclusion.

Look up `scipy.stats.shapiro` for usage and details.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.shapiro.html

In [14]:
```
from scipy import stats
res = stats.shapiro(c)
print(f'''The p-value = {res.pvalue} > 0.05 therefore we fail to reject the null hypothesis
      and conclude that c comes from a normal distribution''')
```

```
The p-value = 0.7866973956692646 > 0.05 therefore we fail to reject the null hypothesis
      and conclude that c comes from a normal distribution
```

NP.2c:

Identify and print the values in `c` that are more than 3 standard deviations from the mean of `c`. Report the proportion of values that are more than 3 sd from the mean.

Desired output: `[ 54.1 ... 148.3 ]`

`0.32 of the values are beyond 3 sd from the mean.`

In [15]:
```
stand_c = (c - c.mean()) / c.std()
std_3 = c[(stand_c > 3) | (stand_c < -3)]

print(std_3)
print(f'''{len(std_3)/len(stand_c)} of the values are beyond 3 sd from the mean.''')
```

```
[146.5  53.8  46.8 156.3]
0.004 of the values are beyond 3 sd from the mean.
```

NP.3:

Task: Make a 3x3 identity matrix called `I3`

Desired output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [16]:
```
I3 = np.array([[1.0, 0, 0], [0, 1.0, 0], [0, 0, 1.0]])
print(I3)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

NP.4a:

Task: Make a 10x10 array of values 1 to 100. Call it X

Desired output:

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
```

In [17]:
```
X = np.arange(1, 101).reshape((10,10))
print(X)
```

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
```

NP.4b:

Task: Make a copy of X, call it Y (1 line). Replace all values in Y that are not squares with 0 (1 line). see `numpy.isin()`

Desired output:

```
[[  1   0   0   4   0   0   0   0   9   0]
 [  0   0   0   0   0  16   0   0   0   0]
 [  0   0   0   0  25   0   0   0   0   0]
...
 [  0   0   0   0   0   0   0   0   0 100]]
```

In [18]:
```
Y = X.copy()
Y[np.isin(Y, [1, 4, 9, 16, 25, 36, 49, 64, 81, 100], invert=True)] = 0
print(Y)
```

```
[[  1   0   0   4   0   0   0   0   9   0]
 [  0   0   0   0   0  16   0   0   0   0]
 [  0   0   0   0  25   0   0   0   0   0]
 [  0   0   0   0   0  36   0   0   0   0]
 [  0   0   0   0   0   0   0   0  49   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [  0   0   0  64   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [ 81   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0 100]]
```

NP.5:

Task: Use `np.tile()` to tile a 2x2 diagonal matrix of integers to make a checkerboard pattern. Call the matrix `checkers`

Desired output:

```
[[1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 ...
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]]
```

In [19]:
```
two_diag_mat = np.array([[1, 0], [0, 1]])
checkers = np.tile(two_diag_mat, (4,4))
print(checkers)
```

```
[[1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]]
```

NP.6:

Task: convert the values in `f_temp` from Farenheit to celsius. The conversion is subtract 32, then multiply by 5/9. Round to 1 decimal place.

Desired output:

```
[[21.1 21.7 22.2 22.8 23.3 23.9]
 ...
 [34.4 35.  35.6 36.1 36.7 37.2]]
```

In [20]:
```
# do not modify
f_temp  = np.arange(70,100).reshape((5,6))
```

In [21]:
```
c_temp = ((f_temp - 32)*5/9).round(1)
print(c_temp)
```

```
[[21.1 21.7 22.2 22.8 23.3 23.9]
 [24.4 25.  25.6 26.1 26.7 27.2]
 [27.8 28.3 28.9 29.4 30.  30.6]
 [31.1 31.7 32.2 32.8 33.3 33.9]
 [34.4 35.  35.6 36.1 36.7 37.2]]
```

NP.7:

Task: Convert values in the matrix `x` into z-scores by column, call it matrix `z`. That is: each column should have a mean of 0 and std of 1. (subtract the column mean, and divide by the column std). (not required, but see if you can do this in one line)

Print the column means and column std to show that they have been standardized.

Desired output:

```
[[ 0.41148428  1.04307072 -1.43031951]
 [-0.34069129 -1.14825432 -0.77270135]
...
 [-0.60705751 -1.08536687 -0.57430135]
 [-1.28156585 -0.81250928  1.52877401]]

[-1.16573418e-16  7.77156117e-17  1.44328993e-16]
[1. 1. 1.]
```

In [22]:
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 30).reshape(10,3)
```

In [23]:
```
z = (x - x.mean(axis = 0))/x.std(axis = 0)
print(z)
print(z.mean(axis = 0))
print(z.std(axis=0))
```

```
[[ 0.41148428  1.04307072 -1.43031951]
 [-0.34069129 -1.14825432 -0.77270135]
 [-1.00437562 -1.26512499  0.46033272]
 [ 1.34064235  1.36446505  0.50143385]
 [-1.13711249  0.92620005  1.85777132]
 [ 1.07516861 -1.38199565  0.87134407]
 [ 0.19025617 -0.85607765  0.13152363]
 [ 1.38488797  0.60480571 -0.77270135]
 [-1.53532308  0.42949971 -1.34811724]
 [-0.38493691  0.28341137  0.50143385]]
[-1.16573418e-16  7.77156117e-17  1.44328993e-16]
[1. 1. 1.]
```

NP.8:

Task: Convert values in the matrix `x` into scaled values from 0 to 10. That is take each column and scale values linearly so that the largest value is 10, and the smallest value in the column is 0. Round results to 2 decimal places. Call the result `y`

(Not required, but see if you can do the calculations in one line.)

Desired output:

```
[[0.67 0.88 0.  ]
 [0.41 0.09 0.2 ]
 ...
 [0.   0.66 0.02]
 [0.39 0.61 0.59]]
```

In [24]:
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 30).reshape(10,3)
```

In [25]:
```
y = ((x - x.min(axis=0))/(x - x.min(axis=0)).max(axis=0)).round(2)
print(y)
```

```
[[0.67 0.88 0.  ]
 [0.41 0.09 0.2 ]
 [0.18 0.04 0.57]
 [0.98 1.   0.59]
 [0.14 0.84 1.  ]
 [0.89 0.   0.7 ]
 [0.59 0.19 0.48]
 [1.   0.72 0.2 ]
 [0.   0.66 0.02]
 [0.39 0.61 0.59]]
```

NP.9:

Task: Replace all NaN values in the matrix `x` with 0.

Desired output:

```
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
```

In [26]:
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 12).reshape(4,3).astype('float')
row = np.array([1, 2, 3])
col = np.array([2, 1, 0])
x[row, col] = np.nan
```

In [27]:
```
x[np.isnan(x)] = 0.
print(x)
```

```
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
```